Circuits, Systems, and Signal Processing (2023) 42:780–800
https://doi.org/10.1007/s00034-022-02232-y

Area-, Power-, and Delay-Optimized 2D FIR Filter Architecture for Image Processing Applications

Gundugonti Kishore Kumar[1] · Ravi Raja Akurati[1] · Venkata Hanuma Prasad Reddy[1] · Soumica Cheemalakonda[1] · Sudeeksha Chagarlamudi[1] · Bhasita Dasari[1] · Sameera Sulthana Shaik[1]

Received: 13 April 2022 / Revised: 29 October 2022 / Accepted: 29 October 2022 /
Published online: 11 November 2022
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2022

Abstract

The 2D finite impulse response filters are efficient and are of low complexity, and they are often used for image restoration, image enhancement, and denoising applications. The 2D FIR filter architecture usually includes multipliers implemented using a network of adders/subtractors and shifters, which can be further optimized. Radix-2r multiplication is used for area, power, and delay optimization in the proposed research work. The symmetric 2D FIR filter coefficients considered in the present work are obtained using the modified Park–McClellan transformation method and are optimized using radix-2r multiplication. These coefficients are implemented using fully direct and hybrid II architectures and in gate-level Verilog HDL. The optimized filter's efficacy is checked using Intel DSP Builder and synthesized using the Cadence RTL compiler. The results of both FPGA and ASIC synthesis results are compared to previous studies. The area, power, and delay results show that the proposed filter occu-

✉ Gundugonti Kishore Kumar
gkishorekumar@vrsiddhartha.ac.in

Ravi Raja Akurati
ravirajaakurathi@gmail.com

Venkata Hanuma Prasad Reddy
prasad.rvh@gmail.com

Soumica Cheemalakonda
soumicacheemalakonda@gmail.com

Sudeeksha Chagarlamudi
188W1A04C7@vrsiddhartha.ac.in

Bhasita Dasari
188W1A04D2@vrsiddhartha.ac.in

Sameera Sulthana Shaik
189W5A0421@vrsiddhartha.ac.in

[1] Department of ECE, V R Siddhartha Engineering College, Vijayawada 520007, India

pies less area, consumes less power, and has less delay than existing optimizations. Additionally, this optimization approach is implemented on standard coefficients of 3×3, 5×5, and 7×7 kernels. The FPGA results of these kernels are also compared with previous studies.

Keywords 2D FIR filter · Radix-2r multiplication · FPGA · ASIC · Hybrid II architecture · RTL

1 Introduction

Two-dimensional FIR (finite impulse response) filters are highly applicable across different domains and are especially useful in image processing applications. The high efficiency and low complexity of the 2D FIR filters are favorable for applications like image restoration, enhancement, denoising applications, etc. 2D FIR filter optimization is a technique adapted to make the most effective use of the filter for a specific application.

The area is one of the major concerns in the hardware structure implementation of 2D FIR filters. Most of the symmetric filter structures area is occupied by the multipliers. This issue can be resolved by adapting a multiplier-less filter operation in which the filter coefficients are represented in the form of an optimized binary format, consequently reducing the number of adders/subtractors. Previously, various realization methods and concepts associated with area-efficient filters were presented [14, 15]. Kumar et al. [14] suggested a multiplier-less 2D IIR filter structure for smaller and higher-order filters using distributed arithmetic (DA) technique. Similarly, in [15], distributed arithmetic (DA) algorithm is used to design a systolic architecture for 2D FIR filters. Another effective optimization method was presented in [6], in which symmetric FIR filter design optimization is performed using the coefficients derived from fast converging cuckoo search algorithms.

A study on resource usage of FPGA for different sizes of 2D Gaussian filter to state the differences between implementations of fixed point and the floating point was carried out in [3]. This study has proven that the floating-point approach consumes a more significant number of resources than fixed-point approach. An interesting concept of stationing a 2D convolver on an FPGA divided into static region and partially reconfigurable region, so that the processing of an image can be done according to the image characteristics, is presented in [8].

Rajakumar et al. [20] proposed a design for the generation and degeneration of BMC, BPSC, and PC techniques for single chips. They claimed that the power consumption had been reduced by 33% using any of the techniques mentioned above. A basic hybrid adder using 2-bit adders, BEC, and 4:1 multiplexer is designed in [23]. This design has shown an impressive improvement in delay. Differential evolution (DE) algorithm is used to design a filter with fewer SPT terms in [9]. The shift and add approach is used in this study, which reduced the area and power consumption. To minimize multiple constant multiplications, Gundugonti and Narayanam [11] modified multiplier blocks using modified radix-2r representation and replaced conventional ripple carry adders with a 4:2 compressor.

Recently, a circular symmetric 2D FIR filter architecture was proposed in [16] using the Park–McClellan transformation method. This work focused on presenting an efficient hardware implementation model. The filter coefficients were derived using modified Park–McClellan transformation and then quantized using canonical signed digit (CSD) binary number format, which reduced the number of adders/subtractors using common subexpression elimination (CSE) algorithms. Though the model has proven to be highly efficient compared to the previously proposed models in terms of area, power, and speed, it can be optimized further. Odugu et al. [17] proposed optimization of 2D FIR filter using imprecise multipliers and approximate compressors with acceptable accuracy, which adapted circular symmetry.

In this paper, an area-, power-, and delay-optimized 2D FIR filter architecture is designed using the radix-2r arithmetic multiplication technique. In this approach, the filter coefficients are scaled up to obtain integer values, and then these values are multiplied by radix-2r multiplication constants [18]. This process minimizes the number of additions, reducing the number of adders required for implementing the filter. The reduction in the number of adders will reduce the area. The additional advantage of reducing the adder count is the reduction in the simulation time of the filter. Thus, the delay is also reduced. The proposed structures are implemented in the form of Verilog HDL code [19] for 9×9 2D FIR filter coefficients in direct and hybrid forms, which are considered from [16], and for standard Gaussian filter coefficients of kernel sizes 3×3, 5×5, 7×7 and are synthesized using Cadence RTL compiler. The filter's functionality is tested using Cyclone®V 5CSEMA5F31C6 board, and the board description file is created using Intel DSP Builder, which can be operated on MATLAB command prompt. The synthesis is carried out using the Quartus Standard edition. The area, power, and delay results obtained from ASIC and FPGA implementation are compared with the previously proposed models.

The following are the article's main contributions:

1. Radix-2r multiplication constants, where $r = 3$, replace 2D FIR filter coefficients to minimize partial products.
2. Adders and shifters replace multipliers in the filter architecture. The current approach minimizes the number of adders/subtractors utilized in the architecture, thus reducing the area occupied.
3. This approach also reduces power consumption and delay.

The paper is divided into six sections. Section 2 discusses the 2D FIR filter architecture and coefficients considered for the present work, and Sect. 3 shows the proposed model, that is, radix-8 multiplication, Sect. 4 shows the simulation and functional verification using Intel DSP builder and FPGA implementation, Sect. 5 discusses the synthesis results obtained from ASIC and FPGA implementations, and Sect. 6 concludes the paper.

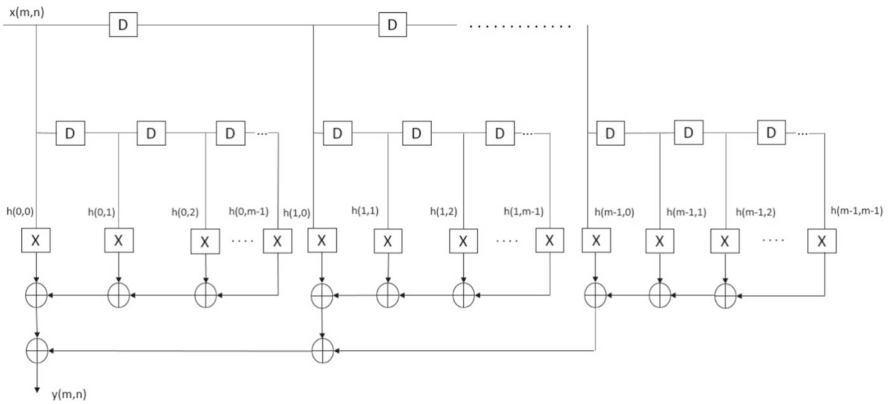


Fig. 1 General architecture of fully direct form

2 2D FIR Filter

2.1 2D FIR Filter Architecture

In this section, the architectures of optimized 2D FIR filters are discussed. There are four different types of 2D FIR filter architectures. They are fully direct form, fully transpose form, transpose–direct form (hybrid I), and direct–transpose form (hybrid II). This work utilizes the architectures for fully direct form and hybrid II form.

The direct form realization is often called a transversal or tapped delay-line filter because the output is just tapping the delayed inputs. This is also called a canonical structure because the number of delay elements used and the order of the filter are the same. The fully direct architecture is shown in Fig. 1.

The hybrid II form, referred to as the direct–transpose form, is a trade-off between the direct form and the transposed form, as the name suggests. The hybrid II form is generally adapted to reduce the critical path, the size of the delay elements, and the fan-out. A hybrid form can be obtained by moving the minimum number of registers from the input path to the accumulation path of the direct form, which satisfies the cycle time requirement. Therefore, it is very suitable for high-speed and low-power applications. The hybrid II architecture is shown in Fig. 2.

2.2 2D FIR Filter Design

Many models have been proposed for the 2D FIR filter designing, such as the ABC algorithm [4], low-complexity circular and fan-type models using tunable farrow structure [2], etc. One of the most efficient models was the filter design discussed in [16], in which the Park McClellan transformation method was used to calculate the filter coefficients. The work focused on designing fully direct and hybrid II filter structures based on the modified filter coefficients, using CSD (canonical signed digit), CSD using horizontal CSE (common subexpression elimination), and vertical CSE techniques.

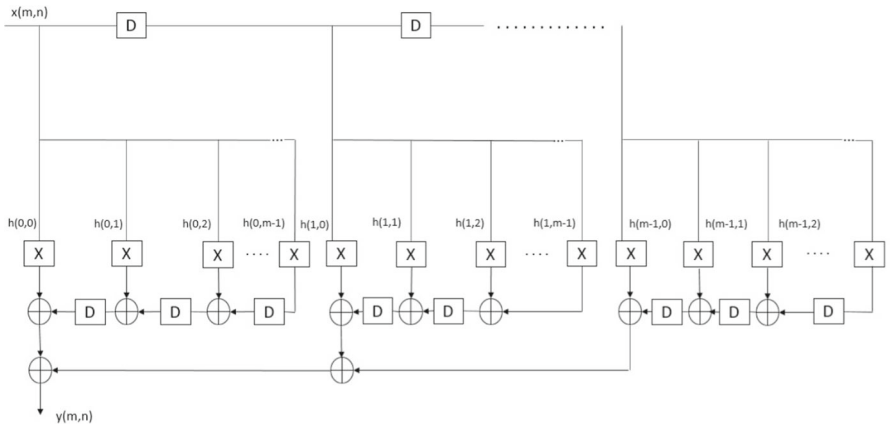


Fig. 2 General architecture of hybrid II form

According to [16], the McClellan transform transforms the 1D filter into a 2D filter. The frequency response 1D FIR filter with length $2N + 1$ and symmetric is given as

$$H(e^{jW}) = h(0) + \sum_{n-1}^{N} h(n)[e^{-jWn} + e^{jWn}] \tag{1}$$

$$= h(0) + \sum_{n-1}^{N} 2h(n) \cos wn \tag{2}$$

The 2D filter frequency response can be obtained from 2 by substituting the terms 1

$$H(e^{jW1}, e^{jW2}) = \sum_{m=0}^{M1} \sum_{n=0}^{M2} a(m,n) \cos mw1 \cos nw2 \tag{3}$$

$$\cos w = \sum_{p=0}^{p} \sum_{q=0}^{q} t(p,q) \cos pw1 \cos qw2 \tag{4}$$

Finally, the response of the 2D FIR filter obtained for filter order $(2NP + 1) \times (2NQ + 1)$ is 1

$$H(e^{jW1}, e^{jW2}) = \sum_{n=0}^{N} b(n) \left[\sum_{q=0}^{q} t(p,q) \cos pw1 \cos qw2 \right]^n \tag{5}$$

The low-pass filter coefficients are designed using the above-mentioned approach for the following specifications considered from Eq. 1. The order of the filter is 9×9, with 16-bit input and output, the stop band attenuation is 29.93 dB, and the pass band ripple is 6.016 dB [16]. For the specifications considered, the filter coefficients are

$$\begin{bmatrix}
-0.0012 & -0.0098 & -0.0034 & -0.0068 & -0.0085 & -0.0068 & -0.0034 & -0.0098 & -0.0012 \\
-0.0097 & -0.0039 & -0.0012 & 0.0007 & 0.0058 & 0.0007 & -0.0012 & -0.0039 & -0.0097 \\
-0.0034 & -0.0051 & 0.0105 & 0.0327 & 0.0415 & 0.0327 & 0.0105 & -0.0051 & -0.0034 \\
-0.0068 & 0.0095 & 0.0327 & 0.0832 & 0.0957 & 0.0832 & 0.0327 & 0.0095 & -0.0068 \\
-0.0085 & 0.0097 & 0.0415 & 0.0957 & 0.1174 & 0.0957 & 0.0415 & 0.0097 & -0.0085 \\
-0.0068 & 0.0095 & 0.0327 & 0.0832 & 0.0957 & 0.0832 & 0.0327 & 0.0095 & -0.0068 \\
-0.0034 & -0.0051 & 0.0105 & 0.0327 & 0.0415 & 0.0327 & 0.0105 & -0.0051 & -0.0034 \\
-0.0097 & -0.0039 & -0.0012 & 0.0007 & 0.0058 & 0.0007 & -0.0012 & -0.0039 & -0.0097 \\
-0.0012 & -0.0098 & -0.0034 & -0.0068 & -0.0085 & -0.0068 & -0.0034 & -0.0098 & -0.0012
\end{bmatrix}$$

Fig. 3 9×9 filter coefficients

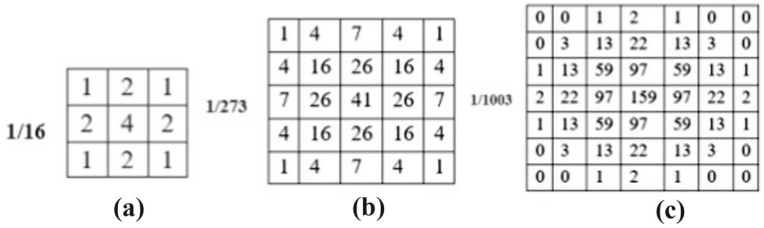


Fig. 4 Approximations of Gaussian kernels of sizes 3×3, 5×5, 7×7

obtained using MATLAB [16] and are implemented using direct and hybrid forms. The coefficients thus obtained are shown in Fig. 3, and the optimization is also carried out for three more kernel sizes, 3×3, 5×5, and 7×7, which are standard Gaussian FIR filters and are shown in Fig. 4.

3 The Proposed Method for Multiplier-Less Filter Operation

Multiplication of data is a fundamental operation in 2D FIR filters. The multiplier block of the filter, where the filter coefficients and inputs are multiplied, has a significant impact on the cost, complexity, and performance. To increase the efficiency of the design, a full-block generic multiplier can be implemented using adders/subtractors. As discussed above, a few techniques are used to optimize the filter architecture, such as CSD, VCSE, and HCSE [16]. The optimization can be further improved using a more efficient multiplication method, radix-2r, for constant multiplication. Radix-2r algorithm is a technique used to synthesize multiplier blocks in order to optimize the design of the 2D FIR filter.

Radix-2r reduces the number of steps involved in performing multiplication compared to conventional methods [13]. Various radix multiplication techniques were discussed, such as radix-2, radix-4, and radix-8, depending on the r value, among which radix-8 can reduce the number of partial products by a factor of 1/3rd and 1/6th when compared to radix-4 and radix-2, respectively [21].

Table 1 Radix-8 booth multiplication table

Bits grouped				P_j
0	0	0	0	0
0	0	0	1	+1
0	0	1	0	+1
0	0	1	1	+2
0	1	0	0	+2
0	1	0	1	+3
0	1	1	0	+3
0	1	1	1	+4
1	0	0	0	−4
1	0	0	1	−3
1	0	1	0	−3
1	0	1	1	−2
1	1	0	0	−2
1	1	0	1	−1
1	1	1	0	−1
1	1	1	1	0

Thus, considering radix-2r multiplication as discussed in [10, 18], the N-bit constant H is given as

$$H = \sum_{j=0}^{(N+1)/r-1} [h_{rj-1} + 2^0 h_{rj} + 2^1 h_{rj+1} + \cdots + 2^{r-2} h_{rj+r-2} - 2^{r-1} h_{rj+r-1}] \tag{6}$$

$$H = \sum_{j=0}^{(N+1)/r-1} P_j X 2^{rj} \tag{7}$$

where N represents the number of bits and P_j in7 represents the value in the booth encoder table corresponding to $r + 1$ bits considered. Here, we consider values of N and r as 12 and 3, respectively. The process groups every $4(r + 1)$ bits of the 2's complement representation of coefficient H and assigns Pj values calculated from6 [13]. The grouping starts from LSB after appending a 0 before LSB.

Thus, for all values of j, the lookup table for the radix-8 multiplier with all P_j values is given in Table 1 [18].

Thus, from 6, 7, and Table 1, the encoding of row filter I of the 9×9 filter is shown in Fig. 5. Since it is a circular symmetric filter, coefficients from $h(0, 5)$ to $h(0, 8)$ will be the same as the coefficients from $h(0, 3)$ to $h(0, 0)$.

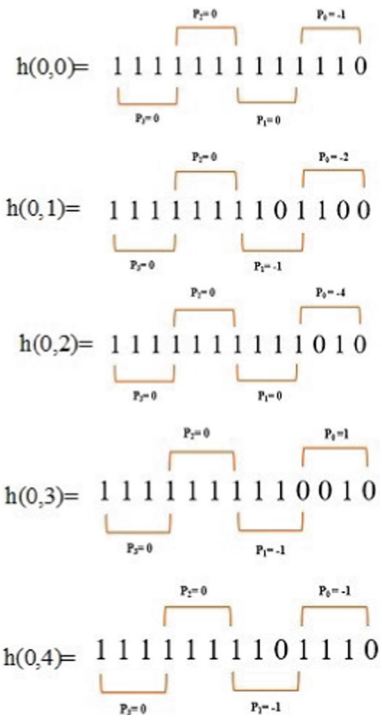


Fig. 5 Encoding of row filter I

This coefficient can thus be obtained from 7, where $N = 12$ and $r = 3$

$$H = \sum_{j=0}^{(N+1)/r-1} P_j X 2^{3j} \tag{8}$$

Thus, by encoding all the coefficients of all the row filters using radix-8 and implementing them in fully direct and hybrid II forms, the number of adders required for the two architectures can be obtained. These adder counts are compared with the corresponding adder counts of direct and hybrid architectures using CSD, HCSE, and VCSE [16] and are shown in Table 2.

The adder count derived from various optimization techniques, comprising direct and hybrid forms for original CSD, CSD with HCSE, and CSD with VCSE, as well as suggested forms, is compared in Table 2. There is a reduction of 19% in the adder count from CSD direct form to proposed direct form and a 24% reduction from CSD hybrid form to proposed hybrid form. The adder count of VCSE architecture is lower than the proposed hybrid form, but the proposed direct form gave a lesser count when compared to VCSE. From this, it can be inferred that adders needed in the proposed radix-8 model are reduced to some extent when compared to other models, and thus the area occupied by the filter architecture and the count of resources needed for its implementation will be reduced, and thus the hardware can be optimized. By implementing the proposed

Table 2 Comparison of adder counts for different optimizations

Row filter number (RF)	Fully direct [16]	Hybrid II [16]	HCSE [16]	VCSE [16]	Proposed Direct	Proposed hybrid
RF-1	14	18	12	16	11	13
RF-2	12	17	12	13	10	11
RF-3	20	27	19	20	15	19
RF-4	20	28	21	18	17	24
RF-5	19	26	18	19	16	23
RF-6	20	28	21	18	17	24
RF-7	20	27	19	20	15	19
RF-8	12	17	12	13	10	11
RF-9	14	18	12	16	11	13
Total	151	206	146	153	122	157

architectures on the FPGA, the hardware utilization results can be obtained, and the area and delay results can be obtained from ASIC implementations.

4 Simulation and Functional Verification

This section discusses the implementation of Verilog HDL of 9×9 coefficients in direct and hybrid form architectures using radix-8 multiplication in DSP Builder, and its functionality is checked on a field-programmable gate array (FPGA).

4.1 DSP Builder

DSP Builder is a program that connects the MATLAB and Simulink software from MathWorks with the Intel Quartus II software from Intel. Automatic Verilog HDL Test Bench creation and Quartus II compilation control are included in the DSP builder. It has a set of fixed-point arithmetic and logical operators that have been used in conjunction with the Simulink software to enable quick prototyping in Intel DSP development boards. SignalTap II Logic analyzer is used to detect signals from Intel device on DSP board and to load the data into MATLAB workspace for analyzing images. Hardware in the loop (HIL) and HDL import enable the Verilog HDL design, and FPGA hardware accelerated co-simulation with Simulink [12].

Figure 6 shows the block diagram describing the steps in developing the DSP Builder and Simulink model file. The Simulink simulates the model (MDL) file, and the compiled block compiles the file to generate output files on which register transfer level (RTL) synthesis is performed. For automated simulation flow, simulation tools like Quartus II software are used. Then, a program is generated that can be loaded into the DE1-SoC hardware development board [12].

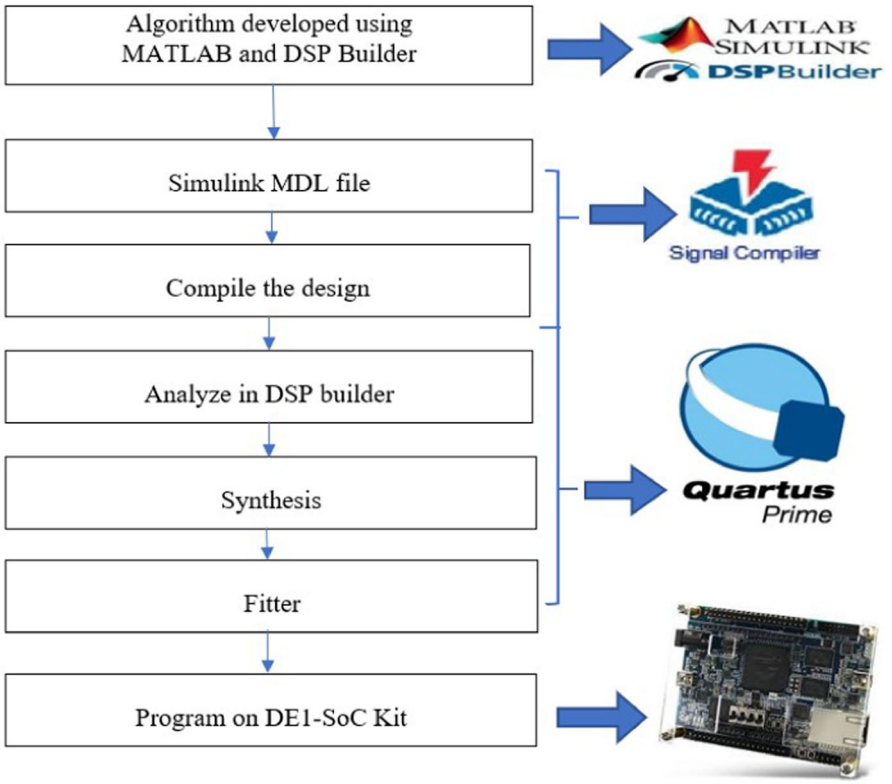


Fig. 6 Block diagram of simulation

The main reason behind using the DSP builder is that it provides built-in-board components for the DE1-SoC board. It contains board libraries, defined by XML board description file, that have all FPGA pin assignments [12]. For this study, the Cyclone®V 5CSEMA5F31C6 board file can be operated on the MATLAB command prompt.

4.2 FPGA Implementation and Verification

This section discusses the hardware implementation and verification of functionality using Intel DSP Builder. Figure 7 shows the simulation setup assembled using DSP Builder. It has an input image block, preprocessing and post-processing blocks, and an output image block. Video viewers are used to viewing the images. The input image is a 512*512 grayscale image. The preprocessing block consists of a 2D-to-1D converter, frame converter, and un-buffer blocks. The post-processing block consists of a data type converter, buffer, and a 1D-to-2D converter. This converts the processed image into a viewable image which can be seen in the video viewer block.

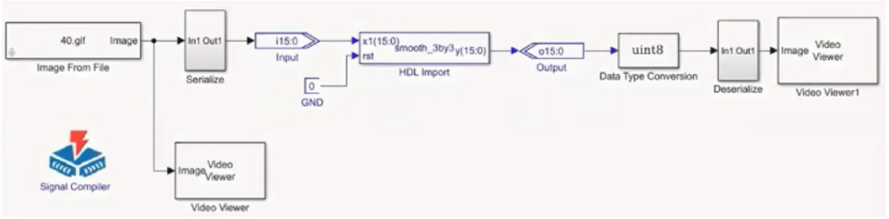


Fig. 7 Software simulation setup using HDL import block

The software simulation using the HDL import block is shown in Fig. 7. The HDL import block consists of 3×3 Verilog HDL code, which is compiled and run. Figure 8a shows the input 512*512 image, and Fig. 8b shows the output.

4.3 FPGA in the Loop (FIL)

The FPGA in the loop (FIL) provides an ideal option for control and system verification without the danger of harm. FIL is a combination of both hardware and software approaches. FPGA-in-the-loop (FIL) simulation provides the facility to use Simulink®or MATLAB®software for testing designs on real hardware for any existing HDL code. The HDL code can be either manually written or software generated from a model subsystem [1]. HDL code is necessary in order to perform FIL simulation. There are two FIL workflows, i.e., existing HDL code (FILWizard) and another is MATLAB code or a Simulink model and an HDL coder license.

Once FIL creates a block or system model, it performs the following processes:

1. A FIL block is created, which represents the HDL code.
2. It provides synthesis, PAR, logical mapping, program file generation, and communications channel.
3. It loads the design onto the FPGA.
4. These features are tuned to the RTL code and developed for a particular board. The block does the following as part of simulation using FIL.
5. Transmits data from Simulink to FPGA.
6. Receives data from FPGA.
7. Implements design in real-time application.

The implementation of FIL consists of two parts, the host system, which is used for simulation and analysis using tools like MATLAB, and the target system, where the algorithm is implemented, such as FPGA.

Figure 9 shows the FIL setup done using the FILWizard command. In the FPGA-in-the-loop command window, select the Altera SoC board and the current FPGA, 5CSEMA5F31C6 and then select the source file and top module, select the reset value and click build. This builds the required FIL block, which consists of a .sof file loaded onto the FPGA board. The input image file is converted into a signal and is given as input from the MATLAB workspace.

Figure 10a shows the FIL output obtained for the 3×3 filter for the input image shown in Fig. 8a. The FIL output for the 5×5 filter is shown in Fig. 10b.

Fig. 8 Input and output images

(a) Input 512*512 image

(b) Output obtained from 3x3 filter using Radix 8

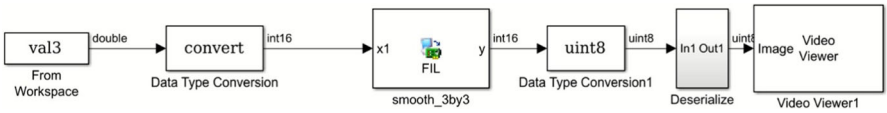


Fig. 9 FIL setup using FILWizard

Similarly, for 7×7 and 9×9 kernels, the output images are obtained by hardware implementation. Lower-order filters are preferable compared to higher-order filters [16]. Though filters with higher orders tend to remove more noise, they add unnecessary artifacts which may blur out the edges of the output image. Therefore, as the order of the filter increases, the output image's clarity degrades.

(a) 3x3 kernel

(b) 5x5 kernel

Fig. 10 Output of FIL implementation

5 Synthesis Results

In this section, the utilization of resources and synthesis results are presented. The 2D FIR filters are implemented in fully direct and hybrid II architectures for four kernel sizes, 3×3, 5×5, 7×7, and 9×9, in Verilog Hardware Description Language (HDL) using canonical signed digit (CSD), and common subexpression elimination (VCSE, HCSE) is considered from [16].

For reconfigurable applications, Cyclone®V 5CSEMA5F31C6 board is considered. Table 3 shows the resource utilization in the FPGA board by the proposed model and the models which used continuous coefficients, CSD, VCSE, and HCSE, which are considered from 11 for 8-bit and 16-bit inputs. Usually, the direct form uses a lesser number of resources when compared to the hybrid form [16]. The table shows that the utilization of resources, LUTs, flip-flops, and slice registers is lower when

Table 3 Comparison of resource utilization by different models on FPGA

Type of multiplier	Type of architecture	8-bit			16-bit		
		LUTs	FFs	SRs	LUTs	FFs	SRs
Continuous coefficients [11]	Fully direct	2020	380	409	2502	472	553
	Hybrid II	2032	577	556	2596	696	663
CSD [11]	Fully direct	614	318	390	2502	970	984
	Hybrid II	628	360	401	2590	976	1058
CSD with HCSE [11]	Fully direct	549	315	328	1965	851	883
	Hybrid II	556	353	366	1973	869	954
CSD with VCSE [11]	Fully direct	577	326	389	1986	847	856
	Hybrid II	658	367	397	2003	880	867
Radix-8	Fully direct	531	321	243	2440	337	262
	Hybrid II	566	347	302	2516	606	524

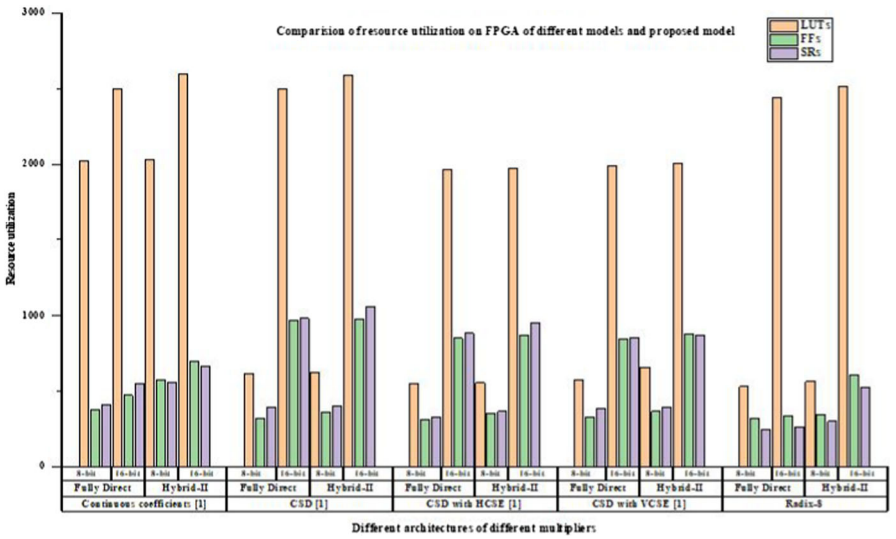


Fig. 11 Graphical comparison of resource utilization for all models on FPGA

compared to other models for the same input bit width. Though the resources utilized by CSE models are lesser than the proposed model for 16-bit input, the decrease in adder count can be observed. As the input bit width decreases, resource utilization decreases but complexity increases [16]. Figure 11 shows the graphical comparison of resource utilization for all models on FPGA for both 8-bit and 16-bit inputs, which is easier to understand. Color coding for different resources is shown below the graph in the figure. It is seen that for the proposed model, except for LUTs, all other resources are moderately used.

The application-specific integrated circuit (ASIC) synthesis results in UMC 90 nm technology are obtained using Cadence RTL Compiler for filter order 9 with 8- and 16-bit inputs and 16-bit output. Figure 12 shows the block diagram representing ASIC implementation of the proposed model in fully direct form. The block is named after the top module used in Verilog HDL code with 8-bit data input, clock, and reset as input wires and 16-bit output as output wires.

Table 4 compares area, delay, and power between the proposed model and models used in [9, 16] for 16-bit input. It can be seen that the area is reduced for the proposed model, and there is a considerable reduction, especially in the hybrid model. Hybrid II forms have considerably more complex when compared to direct forms [9, 11, 16]. Also, the proposed hybrid form area is approximately a 28% reduction compared to other models. Thus, the proposed model is better. Among all the models, continuous coefficients consumed a large amount of power, while other models consumed less power and did not show any significant difference. There is a drop of 50% in delay between continuous coefficients and the proposed model and approximately 25–30% drop between other models and the proposed model.

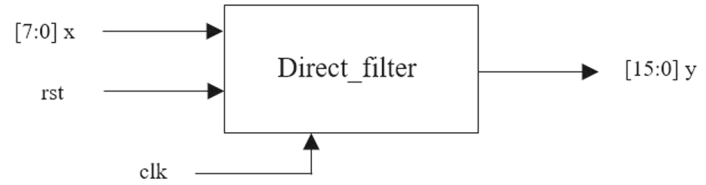


Fig. 12 Block diagram of ASIC implementation

Table 4 Comparison of area, delay, and power of 2D FIR filter of different models in ASIC platform

Type of multiplier	Type of architecture	16-bit		
		Area (μm^2)	Power (mW)	Delay (ns)
Continuous coefficients [16]	Fully direct	1,650,919	244.8	16.01
	Hybrid II	1,640,258	244.03	16.09
CSD [9]	Fully direct	141,530	24.65	14.723
	Hybrid II	157,462	24.49	9.379
CSD with HCSE [16]	Fully direct	140,274	24.57	14.6
	Hybrid II	155,116	25.16	13.34
CSD with VCSE [16]	Fully direct	140,326	27.61	14.68
	Hybrid II	153,600	28.02	13.45
[9]	Direct	145,156	23.28	23.96
[11]	Direct	162,732	31.16	16
Proposed	Fully direct	136,405	24.55	10.785
	Hybrid II	119,118	21.24	8.3

Figure 13a–c shows the graphical comparison of area, delay, and power of 2D FIR architectures using different models in the ASIC platform, respectively. It shows that the area and delay used by the proposed model are well optimized.

Table 5 compares the parameters, area, power, and delay of the proposed model and models implemented in [5, 16] for 16-bit input and the same order of the filter, 9×9. As it can be seen, there is around a 30% decrease in the area of the proposed hybrid model compared to hybrid models of HCSE, VCSE, and the model of Chen et al. Areas of direct form architectures did not show any significant difference, so as power. There is a 26% reduction in the delay between direct forms of HCSE and VCSE and the proposed model. Furthermore, there is a 38% decrease in hybrid forms. From the Chen et al. model to the proposed direct model, the delay was decreased by 32%.

The graphical comparison of synthesis results in Table 5 is shown in Fig. 14a, b. The graphs of area and delay are presented below. Figure 15 shows the layout of the proposed work, which gives an idea of the area occupied by the proposed model.

This implementation is carried out on three other kernels of sizes 3×3, 5×5, and 7×7 implemented in fully direct form, with synthesis results in Table 6. The simulation of the proposed model for different kernel sizes is carried out using Quartus 18.0 supported by DSP Builder. The resources utilized by the proposed filter are

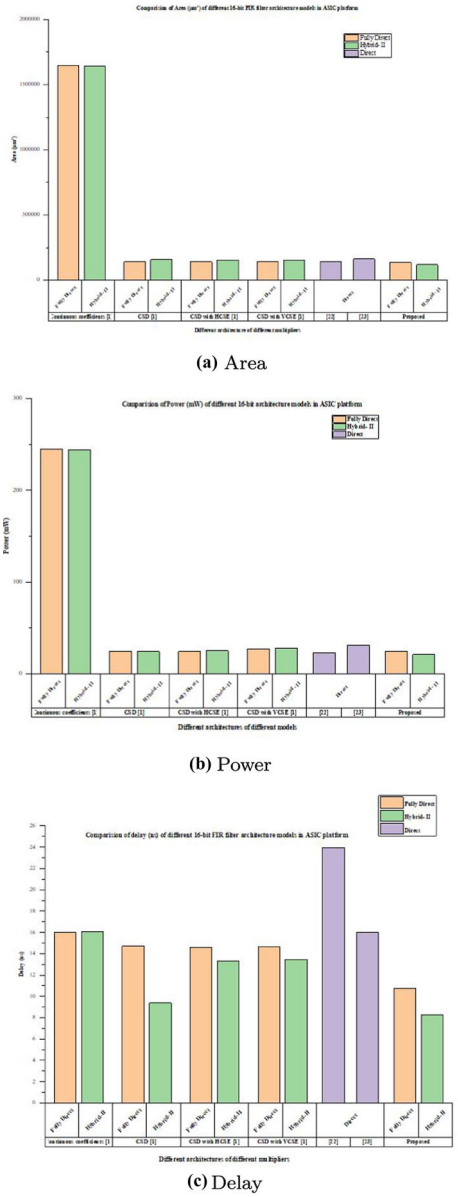


Fig. 13 Graphical comparison of different 16-bit FIR filter architecture models

compared with those of other filters considered from [3, 7, 8]. Dehghani et al. [7] presented an area-efficient reconfigurable convolver implemented on Xilinx Virtex-4 (XC4VLX25) FPGA platform. Fons et al. [8] proposed splitting of FPGA into a partially reconfigurable region that can adapt to arbitrary kernel sizes using Xilinx XCV4LX25 FPGA. Furthermore, Cabello et al. [3] implemented a 2D Gaussian filter using fixed-point and floating-point arithmetic for efficiency using Xilinx XC6SLX16.

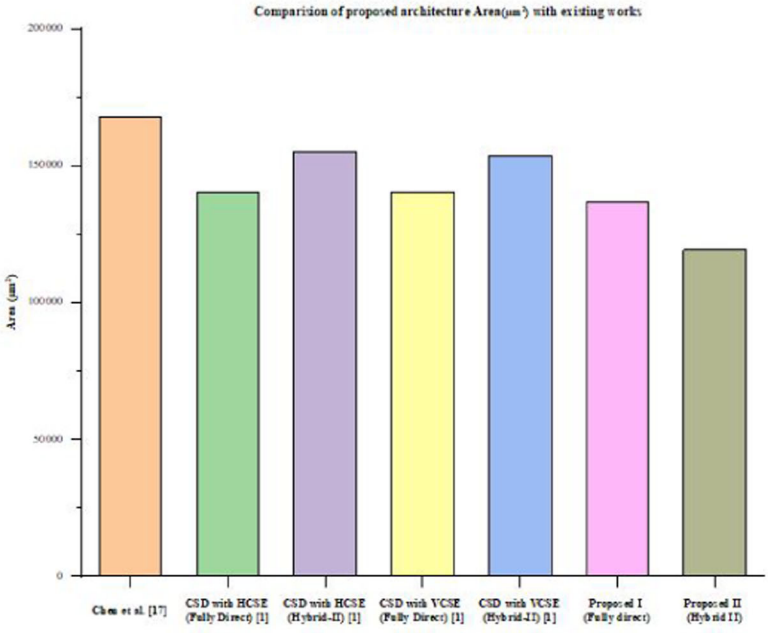
Table 5 Comparison of synthesis results with other works with the same filter order

Architecture	Order	Area (μm^2)	Power (mW)	Delay (ns)
[5]	9	167,778	35.66	12.30
CSD with HCSE (fully direct) [16]	9	140,274	24.57	14.6
CSD with HCSE (hybrid II) [16]	9	155,116	25.16	13.34
CSD with VCSE (fully direct) [16]	9	140,326	27.61	14.68
CSD with VCSE (hybrid II) [16]	9	153,600	28.02	13.45
Proposed I (fully direct)	9	136,405	24.55	10.785
Proposed II (hybrid II)	9	119,118	21.24	8.3

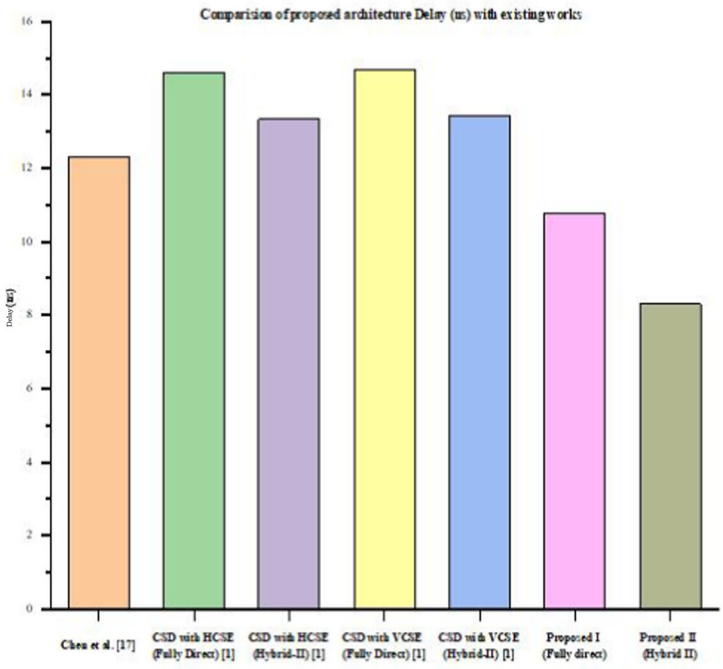
This comparison shows that the proposed model, which used radix-8 multiplication, utilized fewer resources such as flip-flops and LUTs. Between [3, 7] and the suggested one, the LUTs of the 3×3 filter have decreased by 82% and 56%, respectively. Similarly, between [3, 8] and the proposed model, the LUTs of the 5×5 filter have decreased by 94% and 88%, respectively. Furthermore, between [3, 8] and the proposed model, the LUTs of the 7×7 filter have decreased by 89% and 86%, respectively. The adders' count increases as the order of the filter increases; hence, the resources used increase as well; however, they are significantly less when compared to other models of the same order.

6 Conclusion

This study proposes an optimization technique called radix-2r multiplication to optimize the area, power, and delay of 2D FIR filters implemented in fully direct and l-II forms. For this implementation, four kernel sizes, 3×3, 5×5, 7×7, and 9×9, were considered in which 3×3, 5×5, and 7×7 kernels are standard Gaussian filter coefficients, whereas 9×9 filter coefficients are considered from [16], which used Park McClellan transformation to design the filter. The optimization is carried out using radix-8 multiplication. This approach reduced the number of adders required, and thus, the area is reduced. The synthesis is carried out using DSP Builder, into which the Verilog HDL of different filter orders implemented in radix-8 is loaded. A 512*512 image is given as input, and output is obtained through software simulation. Later, the FIL block is built for the FPGA, Cyclone®V 5CSEMA5F31C6, and hardware output is obtained. The FPGA and ASIC results of the proposed model's resources are obtained and compared with the existing works. It can be inferred that the proposed design utilizes fewer resources, such as flip-flops, LUTs, and Slice registers, and uses less area than other models. It is also evident that the proposed model's latency is much lesser than other models. For the rest of the kernels, the synthesis results are compared with the results of [3, 7, 8], and it is seen that the utilization of resources by the filters can be reduced by implementing their architectures using radix-8 multiplication.



(a) Area



(b) Delay

Fig. 14 Graphical representation of comparison of **a** area and **b** delay with other works of same order

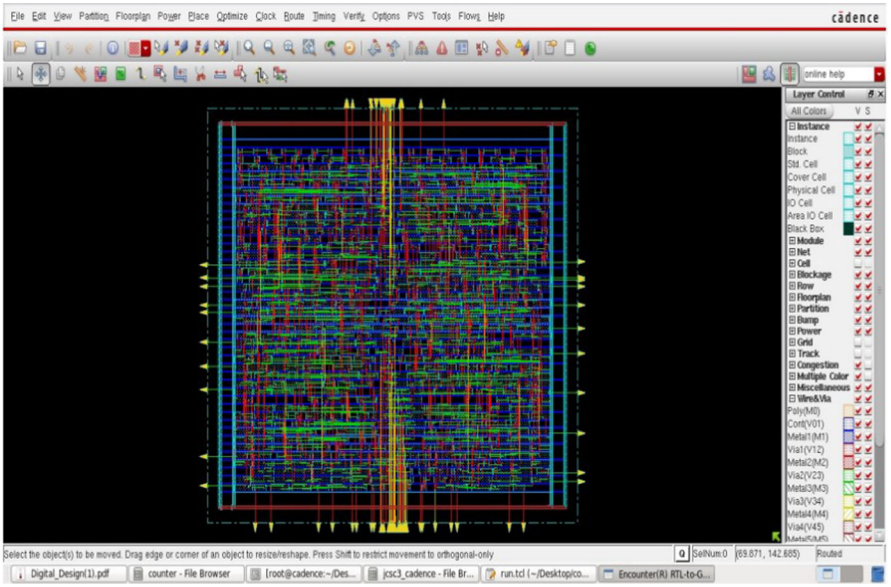


Fig. 15 Layout of the proposed work

Table 6 Comparison of resources utilization of different filters for various kernel sizes

Device	Proposed 5CSEMA5F31C6			[22] XC4VLX25	[8] XCV4LX25		[3] XC6SLX16		
Resource utilization	3 × 3	5 × 5	7 × 7	3 × 3	5 × 5	7 × 7	3 × 3	5 × 5	7 × 7
FF	104	248	394	396	4978	4892	NR	NR	NR
LUT	91	256	358	532	4612	3265	209	2296	2626
DSP Block	0	0	0	14	20	0	0	0	0

Data Availability Data are available on request due to privacy/ethical restrictions.

Declarations

Conflict of interest The authors state that they do not have any conflicts of interest.

References

1. A. Benyamina, S. Moulahoum, FPGA in the loop based single phase power factor correction, in *2019 Progress in Applied Electrical Engineering (PAEE)* (IEEE, 2019), pp. 1–6
2. T. Bindima, E. Elias, Design and implementation of low complexity 2-d variable digital fir filters using single-parameter-tunable 2-d farrow structure. IEEE Trans. Circuits Syst. I Regul. Pap. **65**(2), 618–627 (2017)
3. F. Cabello, J. León, Y. Iano, R. Arthur, Implementation of a fixed-point 2d Gaussian filter for image processing based on FPGA, in *2015 Signal Processing: Algorithms, Architectures, Arrangements, and Applications (SPA)* (IEEE, 2015), pp. 28–33

4. A. Chandra, S. Chattopadhyay, A new strategy of image denoising using multiplier-less fir filter designed with the aid of differential evolution algorithm. Multimed. Tools Appl. **75**(2), 1079–1098 (2016)
5. P.-Y. Chen, L.-D. Van, I.-H. Khoo, H.C. Reddy, C.-T. Lin, Power-efficient and cost-effective 2-d symmetry filter architectures. IEEE Trans. Circuits Syst. I Regul. Pap. **58**(1), 112–125 (2010)
6. P. Das, S.K. Naskar, S. Narayan Patra, Fast converging cuckoo search algorithm to design symmetric FIR filters. Int. J. Comput. Appl. **43**(6), 547–565 (2021)
7. A. Dehghani, A. Kavari, M. Kalbasi, K. RahimiZadeh, A new approach for design of an efficient FPGA-based reconfigurable convolver for image processing. J. Supercomput. **78**(2), 2597–2615 (2022)
8. F. Fons, M. Fons, E. Cantó, Run-time self-reconfigurable 2d convolver for adaptive image processing. Microelectron. J. **42**(1), 204–217 (2011)
9. K.K. Gundugonti, B. Narayanam, An area-power efficient denoising hardware architecture for real EOG signal. J. Circuits Syst. Comput. **29**(15), 2050243 (2020)
10. K.K. Gundugonti, B. Narayanam, Efficient Haar wavelet transform for detecting saccades and blinks in real-time EOG signal. SN Comput. Sci. **2**(3), 1–7 (2021)
11. K.K. Gundugonti, B. Narayanam, High speed fir filter using radix-2 r multiplier and its application for denoising EOG signal. J. Circuits Syst. Comput. **30**(13), 2150237 (2021)
12. N. Habibunnisha, D. Nedumaran, Hardware-based document image thresholding techniques using dsp builder and simulink, in *Artificial Intelligence and Evolutionary Computations in Engineering Systems* (Springer, 2022), pp. 207–220
13. S. Kaur, M.S.M. Suman, S. Manna, Implementation of modified booth algorithm (radix 4) and its comparison with booth algorithm (radix-2). Adv. Electron. Electr. Eng. **3**(6), 683–690 (2013)
14. P. Kumar, P.C. Shrivastava, M. Tiwari, A. Dhawan, ASIC implementation of area-efficient, high-throughput 2-d IIR filter using distributed arithmetic. Circuits Syst. Signal Process. **37**(7), 2934–2957 (2018)
15. P. Kumar, P.C. Shrivastava, M. Tiwari, G.R. Mishra, High-throughput, area-efficient architecture of 2-d block FIR filter using distributed arithmetic algorithm. Circuits Syst. Signal Process. **38**(3), 1099–1113 (2019)
16. V.K. Odugu, C. Venkata Narasimhulu, K. Satya Prasad, Design and implementation of low complexity circularly symmetric 2d FIR filter architectures. Multidimens. Syst. Signal Process. **31**(4), 1385–1410 (2020)
17. V.K. Odugu, C. Venkata Narasimhulu, K. Satya Prasad, An efficient VLSI architecture of 2-d finite impulse response filter using enhanced approximate compressor circuits. Int. J. Circuit Theory Appl. **49**(11), 3653–3668 (2021)
18. A.K. Oudjida, N. Chaillet, Radix-2^r arithmetic for multiplication by a constant. IEEE Trans. Circuits Syst. II Express Briefs **61**(5), 349–353 (2014)
19. S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, vol. 1 (Prentice Hall Professional, Hoboken, 2003)
20. G. Rajakumar, A. Andrew Roobert, T. Arun Samuel, D. Gracia Nirmala Rani, Low power VLSI architecture design of BMC, BPSC and PC schemes. Analog Integr. Circuits Signal Process. **93**(1), 169–178 (2017)
21. P.-M. Seidel, L.D. McFearin, D.W. Matula, Secondary radix recodings for higher radix multipliers. IEEE Trans. Comput. **54**(2), 111–123 (2005)
22. O. Shipitko, A. Grigoryev, Gaussian filtering for fpga based image processing with high-level synthesis tools, in *Proceedings of the IV International Conference on Information Technology and Nanotechnology, Sarma, Russia* (2018), pp. 24–27
23. V. Thamizharasan, N. Kasthuri, FPGA implementation of high performance digital FIR filter design using a hybrid adder and multiplier. Int. J. Electron. 1–21 (2022)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.